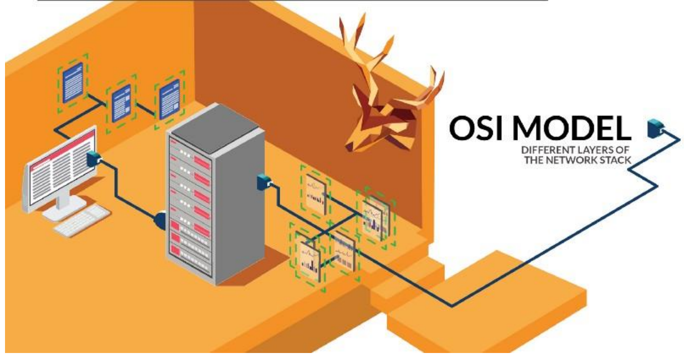


HTTP, Wireshark, and Digital Forensics

Part 2: HTTP traffic contains embedded images

Traffic files:
https://1drv.ms/f/c/a674908a5e2e2f2a/IgDzyvW3yHkdSIWBdVAk82DLAafpQ6YrXyVbQ7S0VAAd73Y?e=iWXBJV

Overview

- Capture HTTP traffic (with a picture embedded)
- Captured HTTP Traffic Overview
- Analyze the second HTTP get request/response
  - Traffic contains the picture
- Extract the building.jpg from the traffic
- Size relation between IP packet and TCP payload

Capture HTTP traffic

with a picture embedded

- Download an image building.jpg
- Save it to /var/www/html

```
/bin/bash 110x21
root@kali:~#
root@kali:~# wget -q https://www.dropbox.com/s/jiq17tgi8u1xls3/building.jpg -P /var/www/html/
root@kali:~#
root@kali:~# ls -l /var/www/html/building.jpg
-rw-r--r-- 1 root root 68661 Oct 26 16:40 /var/www/html/building.jpg
```

-P: directory prefix

Add building.jpg in a webpage image.html

```
root@kali:~# leafpad /var/www/html/image.html
```

image.html

File Edit Search Options Help

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

<img src="building.jpg" alt="UB"/>

</body>
</html>
```

Show the webpage with the image

```
root@kali:~# firefox 127.0.0.1/image.html
```

127.0.0.1/image.html

127.0.0.1/image.html 140%

My First Heading

My first paragraph.

https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/image.html

Capture traffic

/bin/bash 85x7

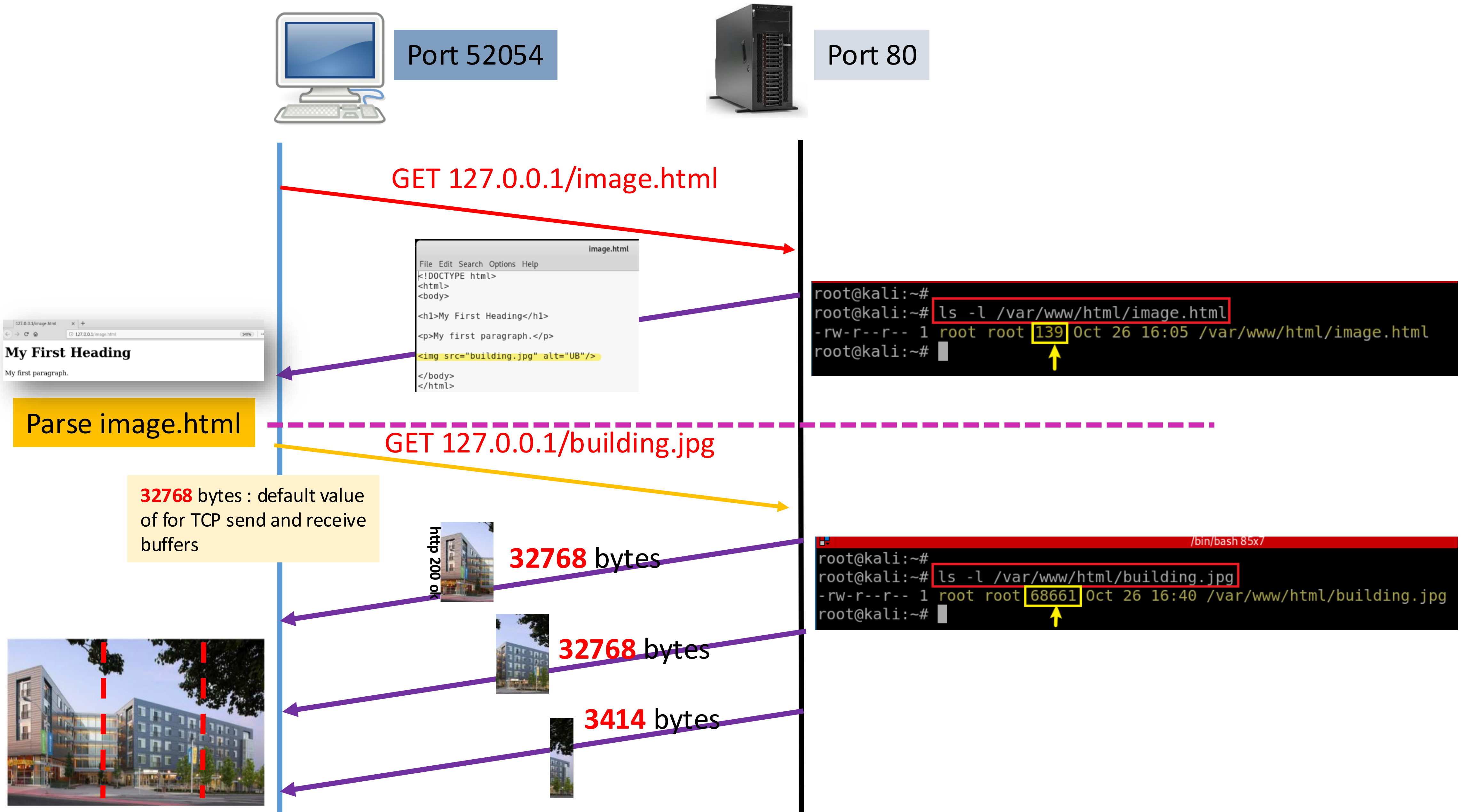
```
root@kali:~# rm -r  ~/.cache/mozilla/firefox/*default*      1
rm: cannot remove '/root/.cache/mozilla/firefox/*default*': N
root@kali:~# wireshark -i lo -k -w ~/traffic/image2.log   2
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_Q
un on Wayland anyway.
^C  5
root@kali:~#
```

/bin/bash 85x14

```
root@kali:~# firefox  127.0.0.1/image.html      3
^CExiting 4 due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.

root@kali:~#
```

HTTP Traffic overview

How many http get requests and responses?



http response head length (289) + Build.jpg (68661) = TCP 1(32768)+TCP 2(32768) +TCP 3 (3414) =68950 wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/image2.log

Verify two HTTP GET: .html and .jpg

```
root@kali:~#
root@kali:~# wireshark traffic/image2.log
```

image2.log

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	52054 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SA
2	0.000008840	127.0.0.1	127.0.0.1	TCP	74	80 → 52054 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 M
3	0.000015234	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=
4	0.386287395	127.0.0.1	127.0.0.1	HTTP	385	GET /image.html HTTP/1.1
5	0.386311944	127.0.0.1	127.0.0.1	TCP	66	80 → 52054 [ACK] Seq=1 Ack=320 Win=65280 Len=0 TSva
6	0.386567159	127.0.0.1	127.0.0.1	HTTP	534	HTTP/1.1 200 OK (text/html)
7	0.386576991	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=320 Ack=469 Win=65152 Len=0 TS
8	0.716114099	127.0.0.1	127.0.0.1	HTTP	346	GET /building.jpg HTTP/1.1
9	0.716125972	127.0.0.1	127.0.0.1	TCP	66	80 → 52054 [ACK] Seq=469 Ack=600 Win=65280 Len=0 TS
10	0.716341907	127.0.0.1	127.0.0.1	TCP	328…	80 → 52054 [ACK] Seq=469 Ack=600 Win=65536 Len=3276
11	0.716353387	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=600 Ack=33237 Win=48512 Len=0
12	0.716363658	127.0.0.1	127.0.0.1	TCP	328…	80 → 52054 [PSH, ACK] Seq=33237 Ack=600 Win=65536 L
13	0.716368424	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=600 Ack=66005 Win=15744 Len=0
14	0.716376090	127.0.0.1	127.0.0.1	HTTP	3480	HTTP/1.1 200 OK (JPEG JFIF image)
15	0.716388599	127.0.0.1	127.0.0.1	TCP	66	[TCP Window Update] 52054 → 80 [ACK] Seq=600 Ack=66
16	0.716445359	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=600 Ack=69419 Win=65536 Len=0
17	0.863058108	127.0.0.1	127.0.0.1	HTTP	307	GET /favicon.ico HTTP/1.1
18	0.863063656	127.0.0.1	127.0.0.1	TCP	66	80 → 52054 [ACK] Seq=69419 Ack=841 Win=65408 Len=0
19	0.863194653	127.0.0.1	127.0.0.1	HTTP	553	HTTP/1.1 404 Not Found (text/html)
20	0.863242007	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=841 Ack=69906 Win=65536 Len=0
21	3.982090205	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [FIN, ACK] Seq=841 Ack=69906 Win=65536 L
22	3.984114989	127.0.0.1	127.0.0.1	TCP	66	80 → 52054 [FIN, ACK] Seq=69906 Ack=842 Win=65536 L
23	3.984119529	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=842 Ack=69907 Win=65536 Len=0

First HTTP GET html

Second HTTP GET .jpg

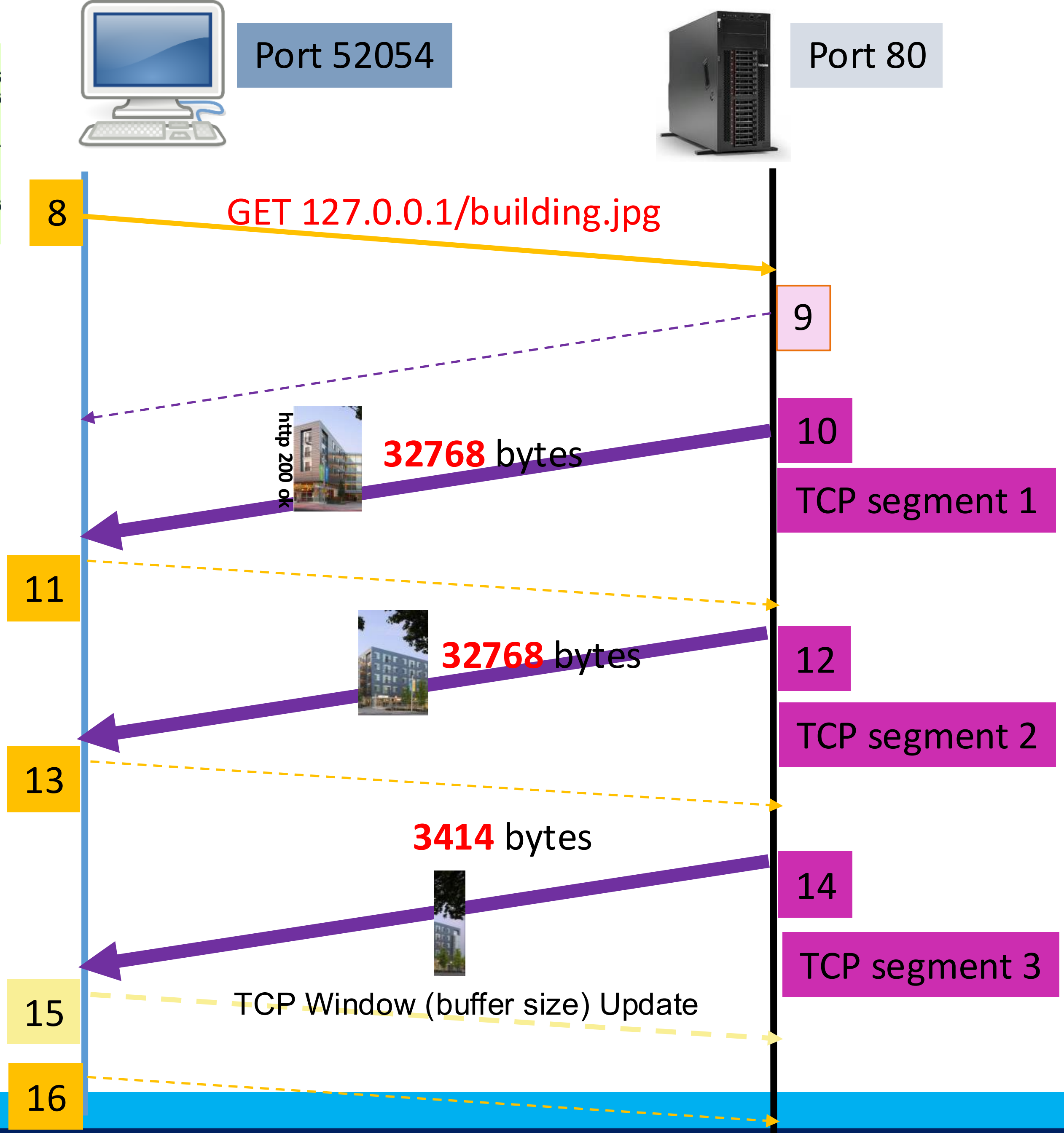
Third HTTP GET .ico

Analyzing the second HTTP get request/response

HTTP traffic for displaying images

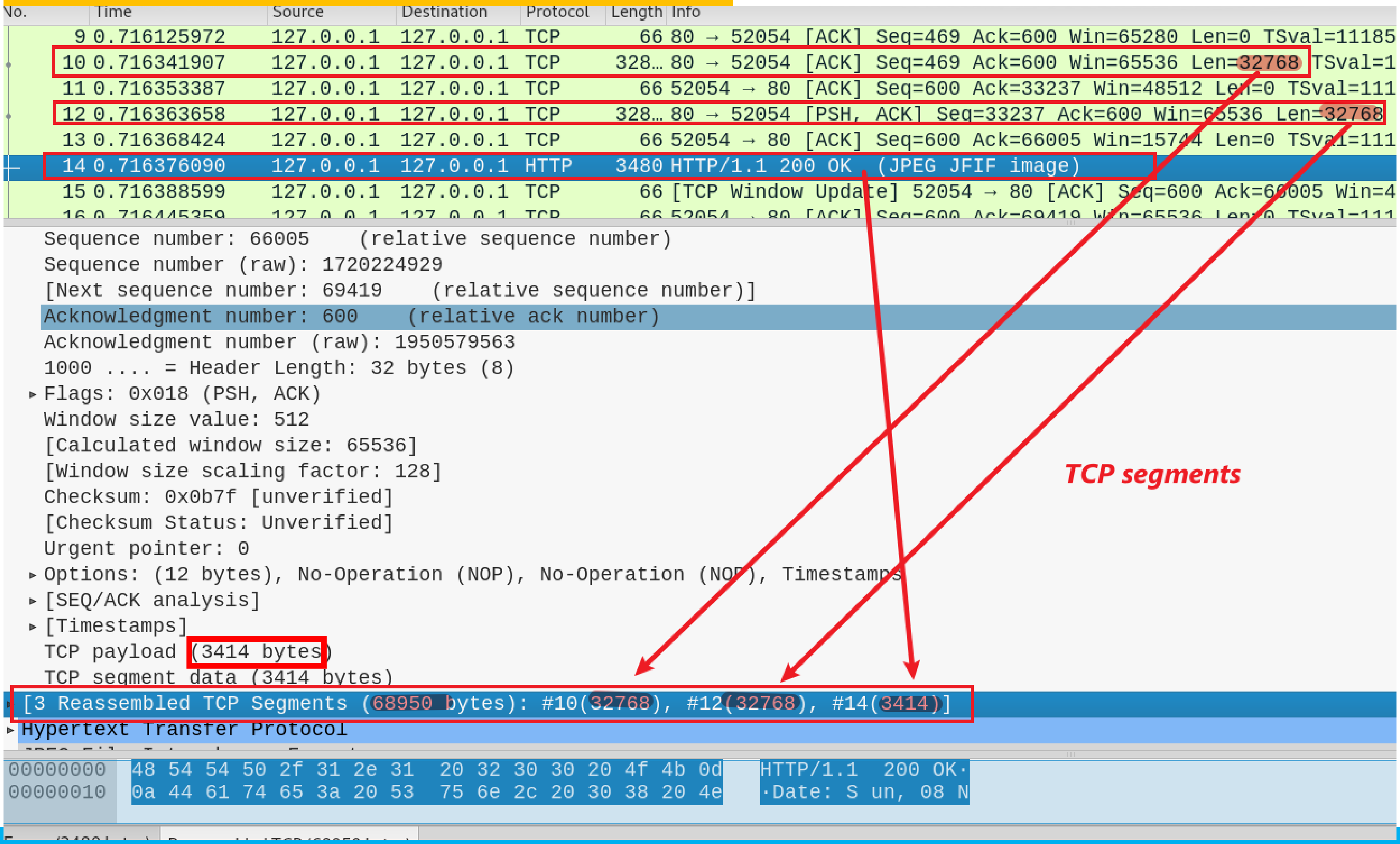
Second HTTP GET .jpg

8	0.716114099	127.0.0.1	127.0.0.1	HTTP	346	GET /building.jpg HTTP/1.1
9	0.716125972	127.0.0.1	127.0.0.1	TCP	66	80 → 52054 [ACK] Seq=469 Ack=600 Win=65280 Len=0 TS
10	0.716341907	127.0.0.1	127.0.0.1	TCP	328…	80 → 52054 [ACK] Seq=469 Ack=600 Win=65536 Len=3276
11	0.716353387	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=600 Ack=33237 Win=48512 Len=0
12	0.716363658	127.0.0.1	127.0.0.1	TCP	328…	80 → 52054 [PSH, ACK] Seq=33237 Ack=600 Win=65536
13	0.716368424	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=600 Ack=66005 Win=15744 Len=0
14	0.716376090	127.0.0.1	127.0.0.1	HTTP	3480	HTTP/1.1 200 OK (JPEG JFIF image)
15	0.716388599	127.0.0.1	127.0.0.1	TCP	66	[TCP Window Update] 52054 → 80 [ACK] Seq=600 Ack=66
16	0.716445359	127.0.0.1	127.0.0.1	TCP	66	52054 → 80 [ACK] Seq=600 Ack=69419 Win=65536 Len=0



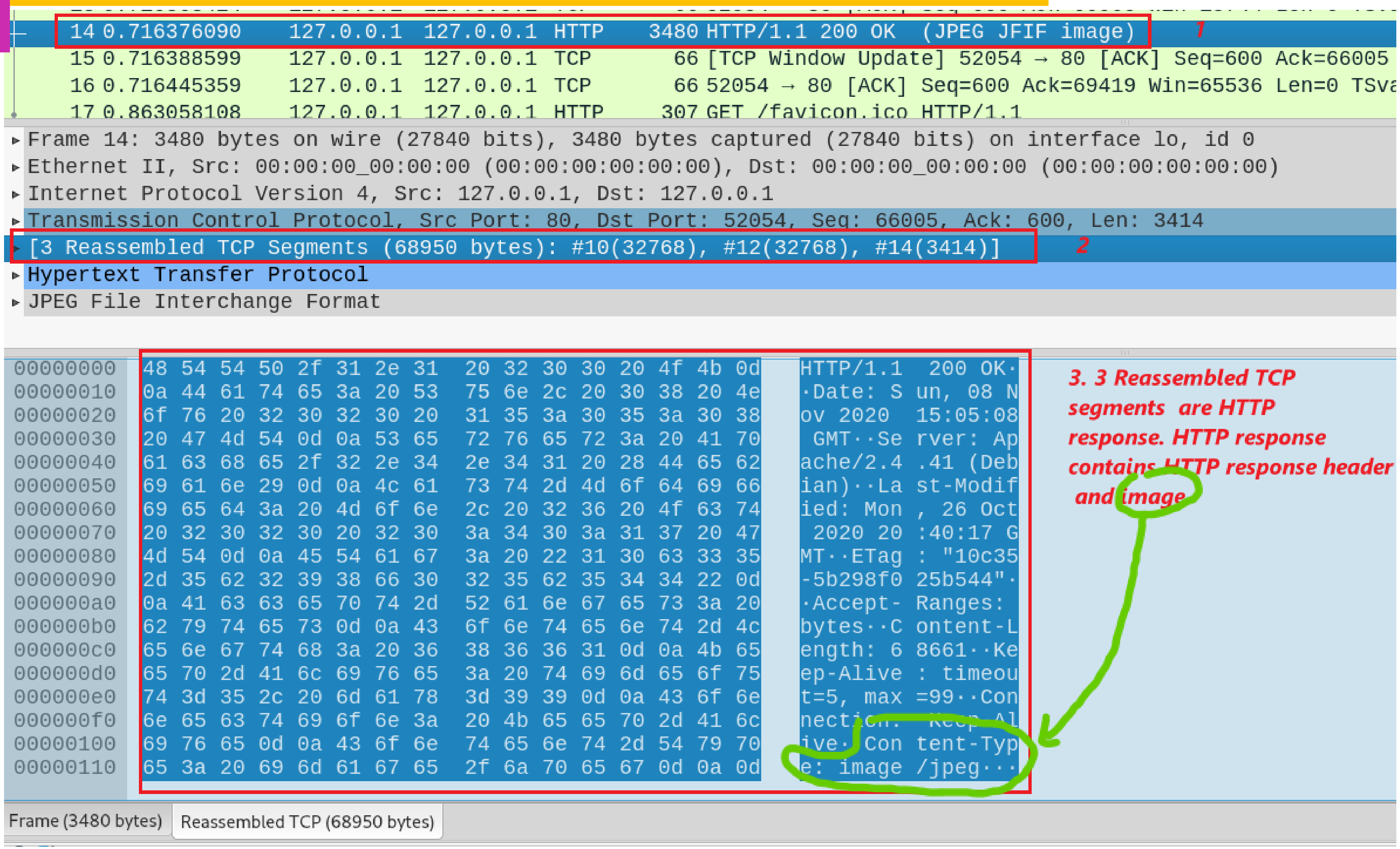
- TCP segment reassembly takes places in HTTP layer.
- TCP reassembling at receiving side
- Wireshark shows the assembled TCP segment at sending side
- TCP segment reassembly works only capture the entire packet and if all the checksums for that packet are valid.

Calculate the length 3 TCP segments: 32768+32768+3414



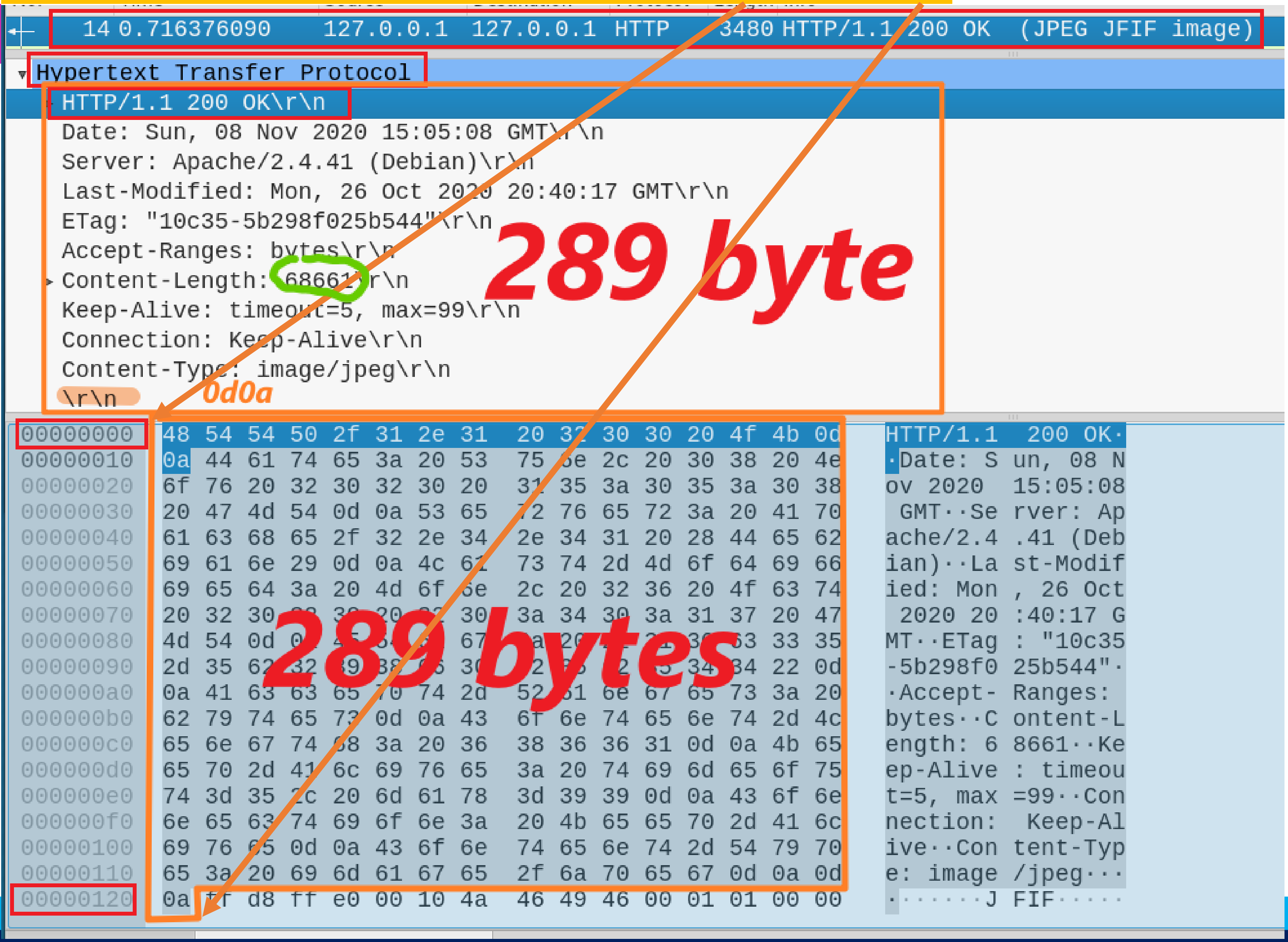
14

Show the content of 3 TCP segments is http response and the data (building.jpg)

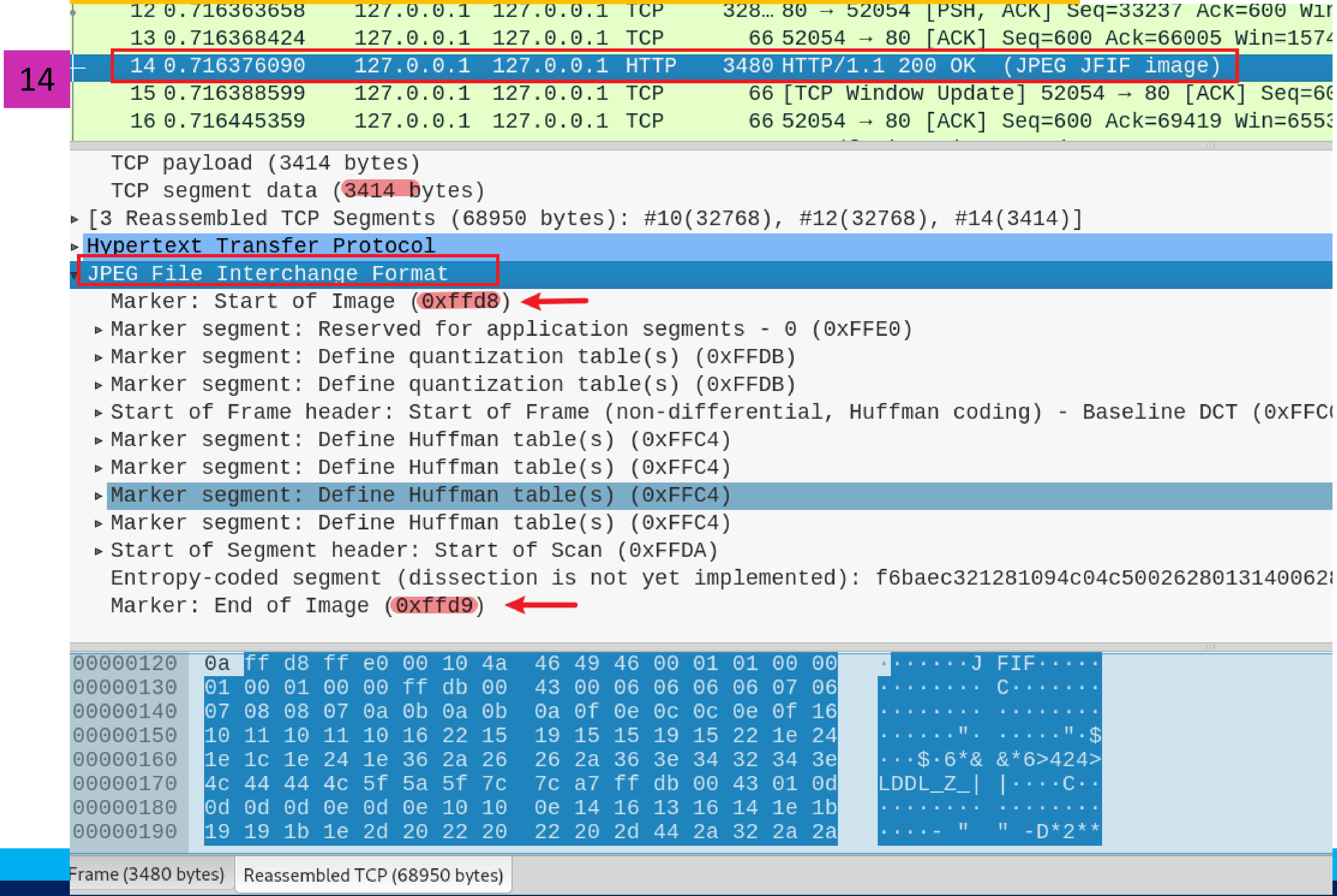


14

Calculate the length of http response header: 289 (0x000 -0x120)

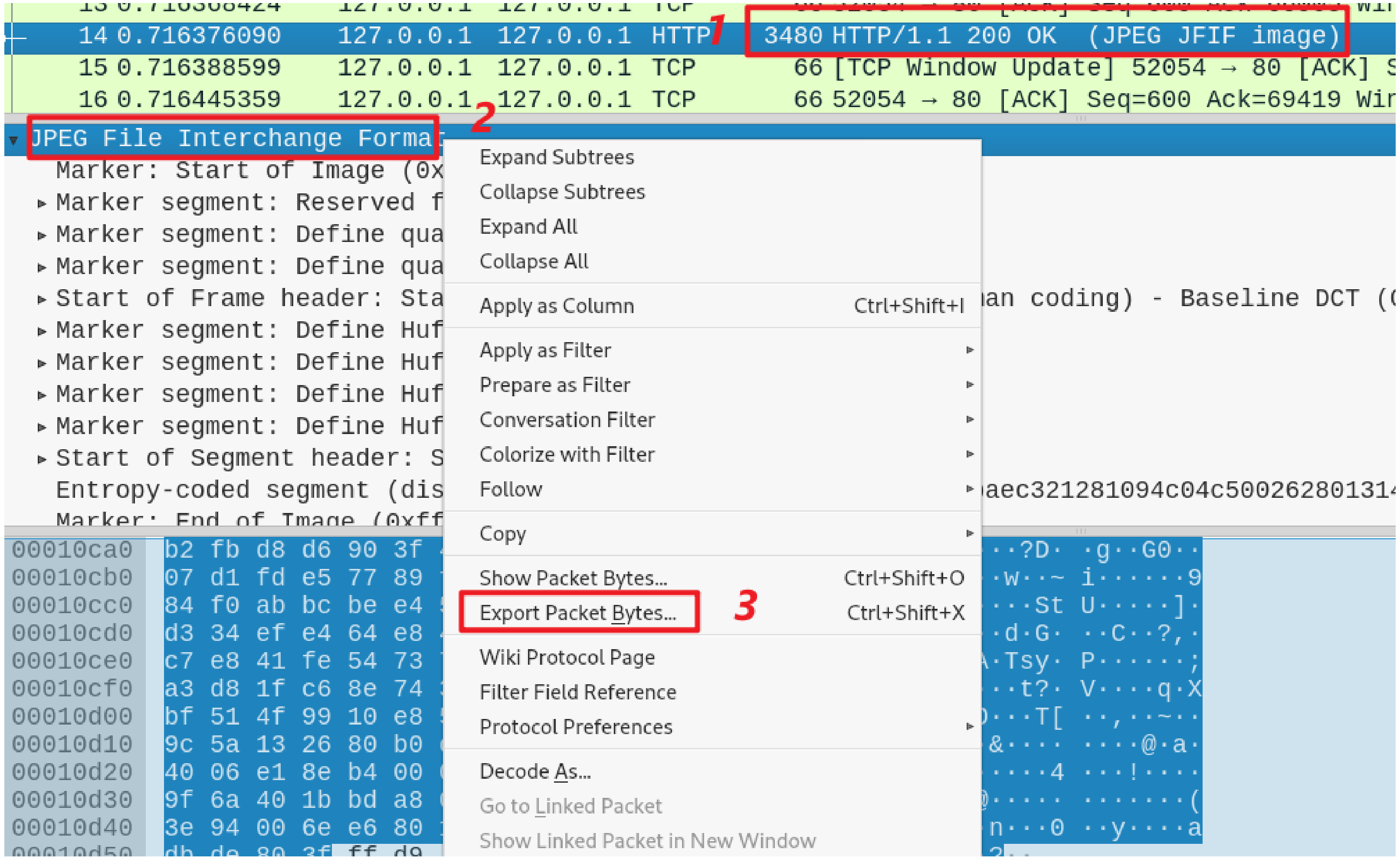


Verify start and end of Image (0x0121 – 0x10d55 +1 =68661 bytes)

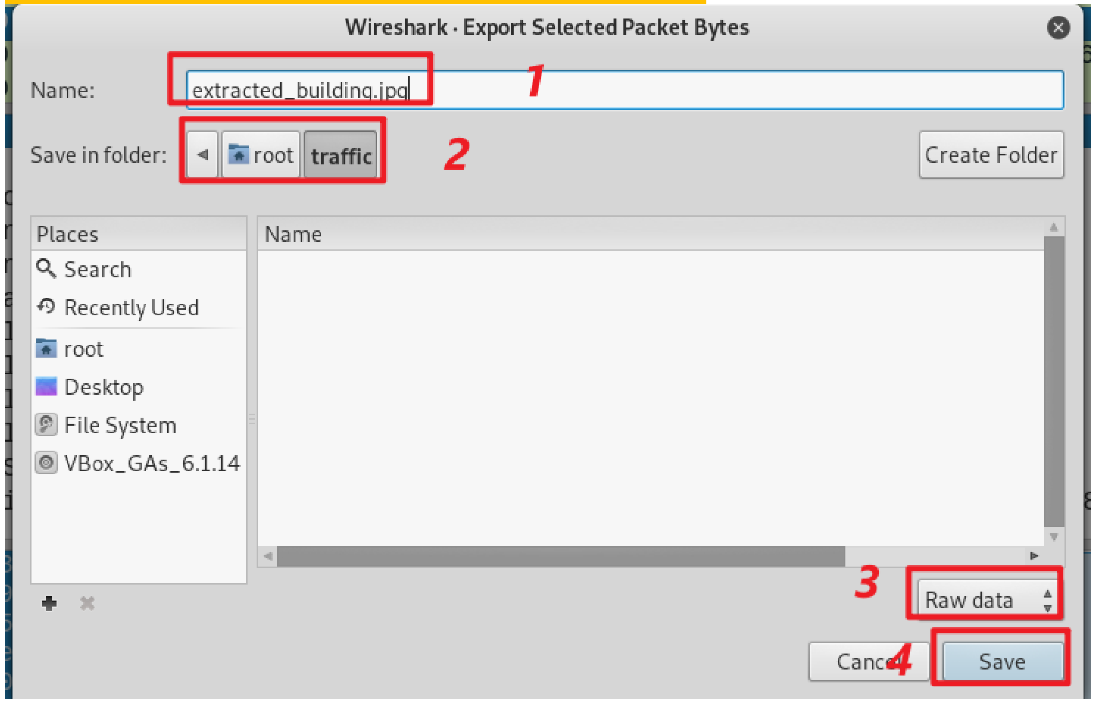


Extract the building.jpg from traffic

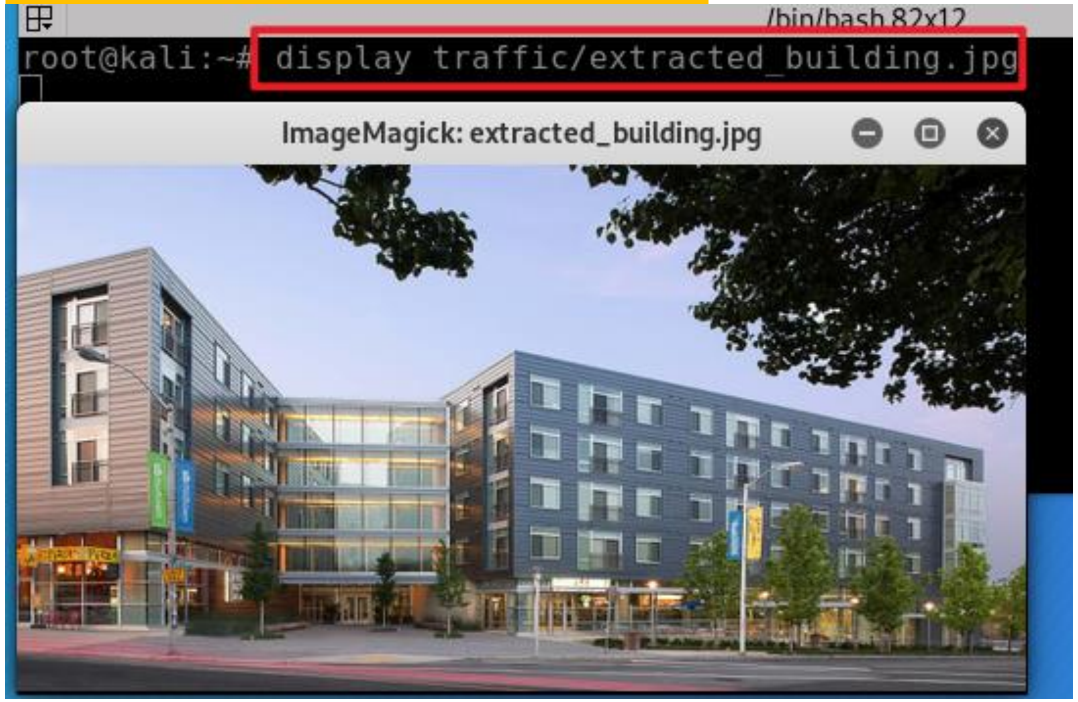
How to extract the building.jpg from log



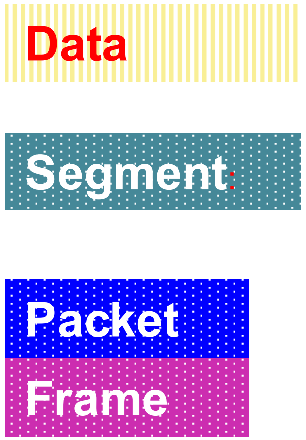
Save extracted image

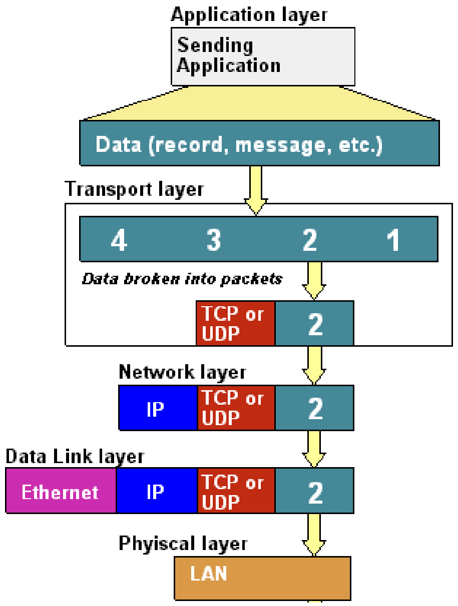


Show image



```
/bin/bash 79x10
root@kali:~#
root@kali:~# ls -l  traffic/extracted_building.jpg
-rw-r--r-- 1 root root 68661 Nov 10 22:27 traffic/extracted_building.jpg
root@kali:~# md5sum  traffic/extracted_building.jpg
442f74691f6ff9ea59036570eba2e183  traffic/extracted_building.jpg
root@kali:~#
root@kali:~# md5sum  /var/www/html/building.jpg
442f74691f6ff9ea59036570eba2e183  /var/www/html/building.jpg
root@kali:~#
```





Size relation between IP packet and TCP payload

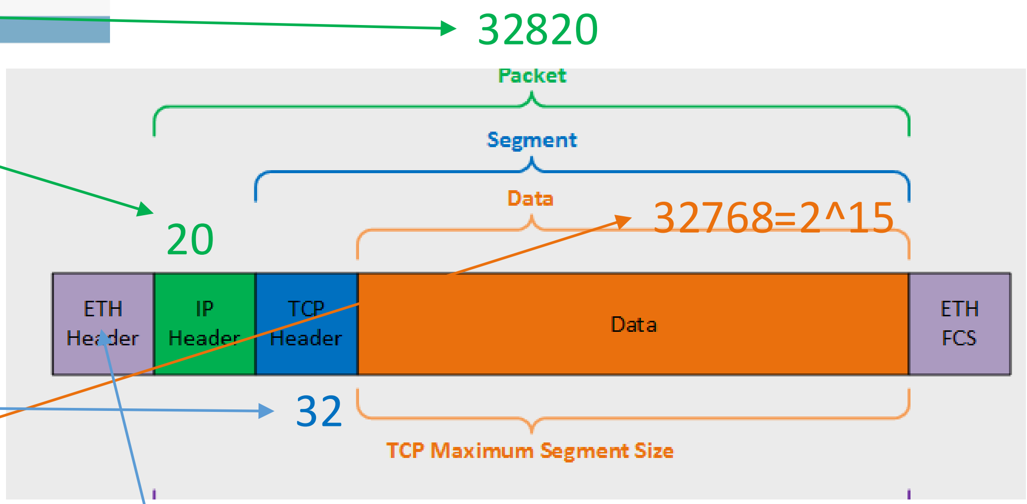
Verify the length relation between IP packet and TCP payload

10

```
10 0.716341907    127.0.0.1  127.0.0.1  TCP     328… 80 → 52054 [ACK] Seq=469 Ack=600 Win=65536 Len=32768 TSva
11 0.716353387    127.0.0.1  127.0.0.1  TCP       66 52054 → 80 [ACK] Seq=600 Ack=33237 Win=48512 Len=0 TSval=
12 0.716363658    127.0.0.1  127.0.0.1  TCP     328… 80 → 52054 [PSH, ACK] Seq=33237 Ack=600 Win=65536 Len=327
13 0.716368424    127.0.0.1  127.0.0.1  TCP       66 52054 → 80 [ACK] Seq=600 Ack=66005 Win=15744 Len=0 TSval=
14 0.716376090    127.0.0.1  127.0.0.1  HTTP    3480 HTTP/1.1 200 OK  (JPEG JFIF image)
```

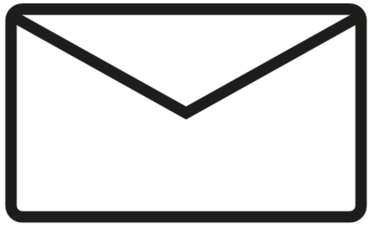
```
▸ Frame 10: 32834 bytes on wire (262672 bits), 32834 bytes captured (262672 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▾ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 32820
    Identification: 0xab19 (43801)
```

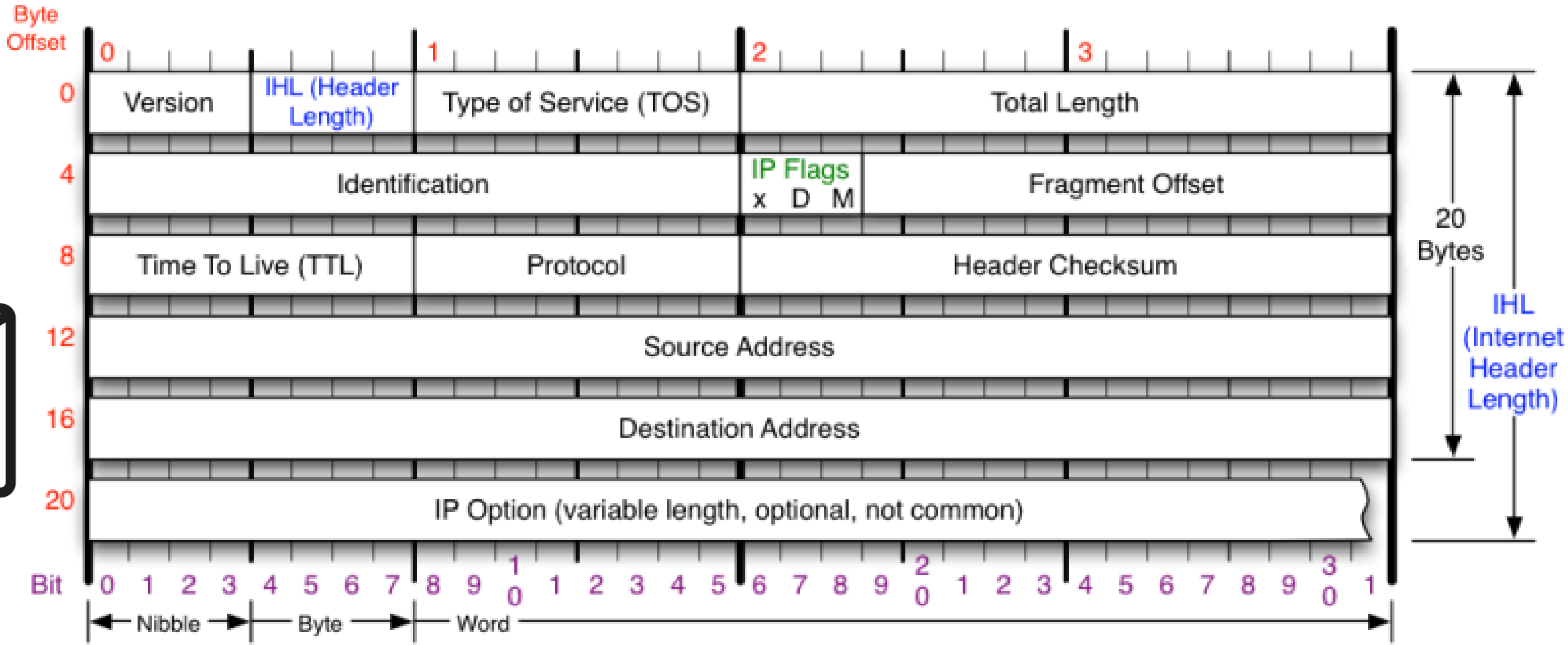
```
▾ Transmission Control Protocol, Src Port: 80, Dst Port: 52054, Seq: 469, Ack: 600, Len: 32768
    Source Port: 80
    Destination Port: 52054
    [Stream index: 0]
    [TCP Segment Len: 32768]
    Sequence number: 469    (relative sequence number)
    Sequence number (raw): 1720159393
    [Next sequence number: 33237    (relative sequence number)]
    Acknowledgment number: 600    (relative ack number)
    Acknowledgment number (raw): 1950579563
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x010 (ACK)
    Window size value: 512
    [Calculated window size: 65536]
    [Window size scaling factor: 128]
    Checksum: 0x7e29 [unverified]
    [Checksum Status: Unverified]
    Urgent pointer: 0
  ▸ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▸ [SEQ/ACK analysis]
  ▸ [Timestamps]
    TCP payload (32768 bytes)
```



TCP segment length needs to be calculated

IP Header





Version

Version of IP Protocol. 4 and 6 are valid. This diagram represents version 4 structure only.

Header Length

Number of 32-bit words in TCP header, minimum value of 5. Multiply by 4 to get byte count.

Protocol

IP Protocol ID. Including (but not limited to):

1 ICMP	17 UDP	57 SKIP
2 IGMP	47 GRE	88 EIGRP
6 TCP	50 ESP	89 OSPF
9 IGRP	51 AH	115 L2TP

Total Length

Total length of IP datagram, or IP fragment if fragmented. Measured in Bytes.

Fragment Offset

Fragment offset from start of IP datagram. Measured in 8 byte (2 words, 64 bits) increments. If IP datagram is fragmented, fragment size (Total Length) must be a multiple of 8 bytes.

Header Checksum

Checksum of entire IP header

IP Flags

x D M

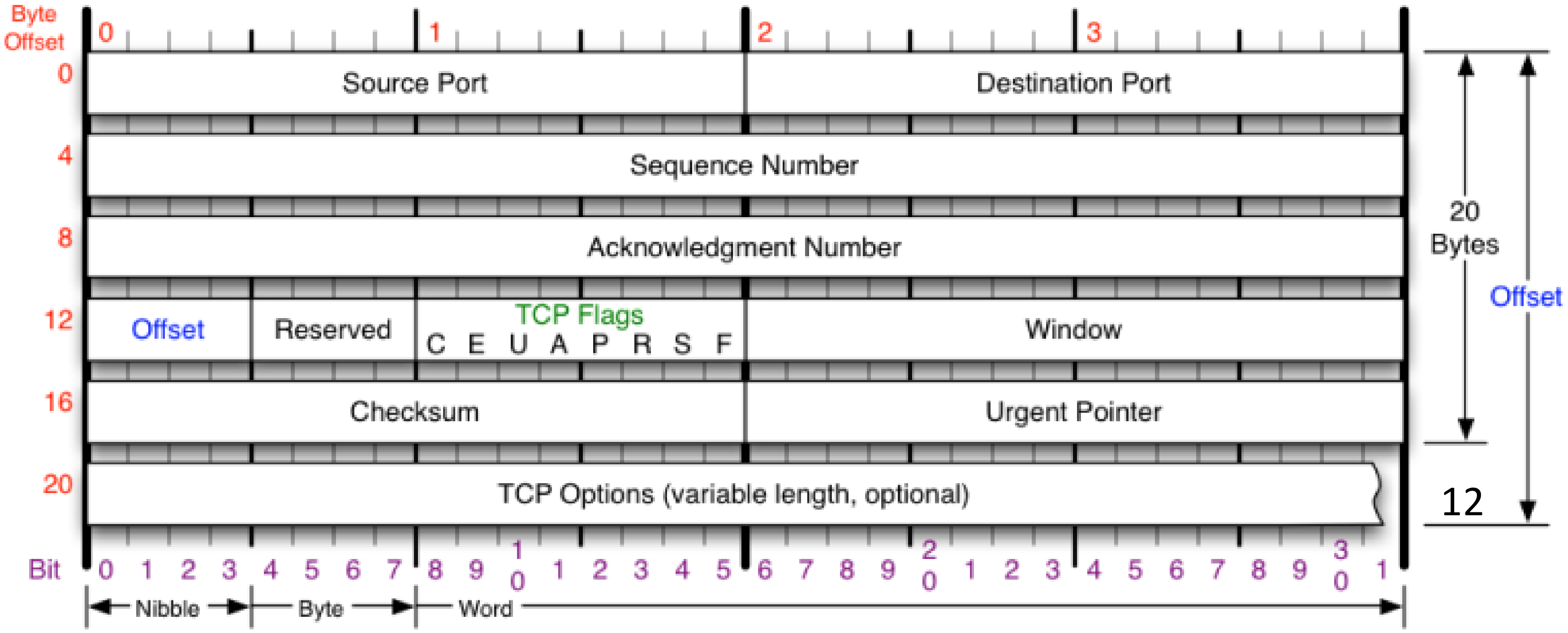
x 0x80 reserved (evil bit)
D 0x40 Do Not Fragment
M 0x20 More Fragments follow

RFC 791

Please refer to RFC 791 for the complete Internet Protocol (IP) Specification.

TCP segment length needs to be calculated

TcP Header



TCP Flags

C E U A P R S F

Congestion Window

- C 0x80 Reduced (CWR)
- E 0x40 ECN Echo (ECE)
- U 0x20 Urgent
- A 0x10 Ack
- P 0x08 Push
- R 0x04 Reset
- S 0x02 Syn
- F 0x01 Fin

Congestion Notification

ECN (Explicit Congestion Notification). See RFC 3168 for full details, valid states below.

Packet State	DSB	ECN bits
Syn	0 0	1 1
Syn-Ack	0 0	0 1
Ack	0 1	0 0
No Congestion	0 1	0 0
No Congestion	1 0	0 0
Congestion	1 1	0 0
Receiver Response	1 1	0 1
Sender Response	1 1	1 1

TCP Options

- 0 End of Options List
- 1 No Operation (NOP, Pad)
- 2 Maximum segment size
- 3 Window Scale
- 4 Selective ACK ok
- 8 Timestamp

Checksum

Checksum of entire TCP segment and pseudo header (parts of IP header)

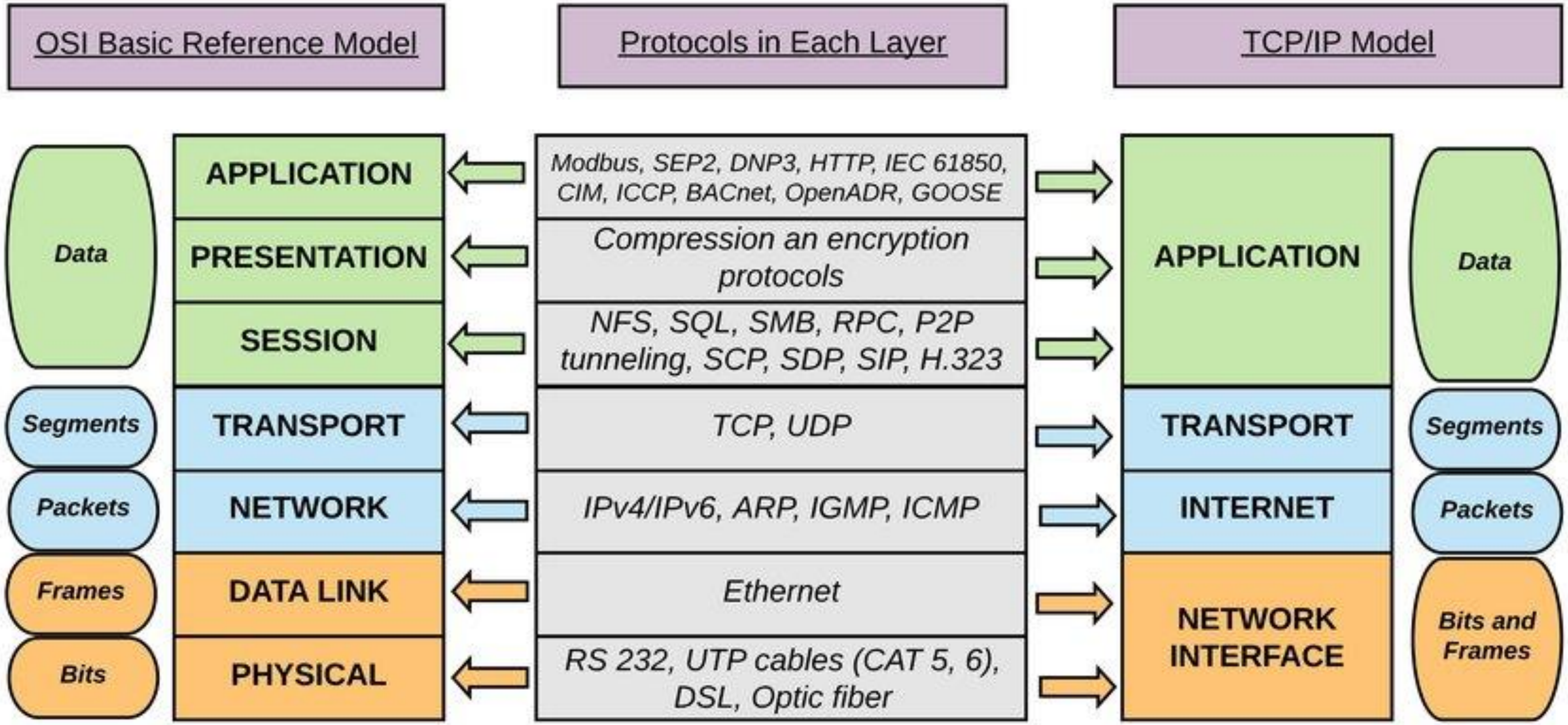
Offset

Number of 32-bit words in TCP header, minimum value of 5. Multiply by 4 to get byte count.

RFC 793

Please refer to RFC 793 for the complete Transmission Control Protocol (TCP) Specification.

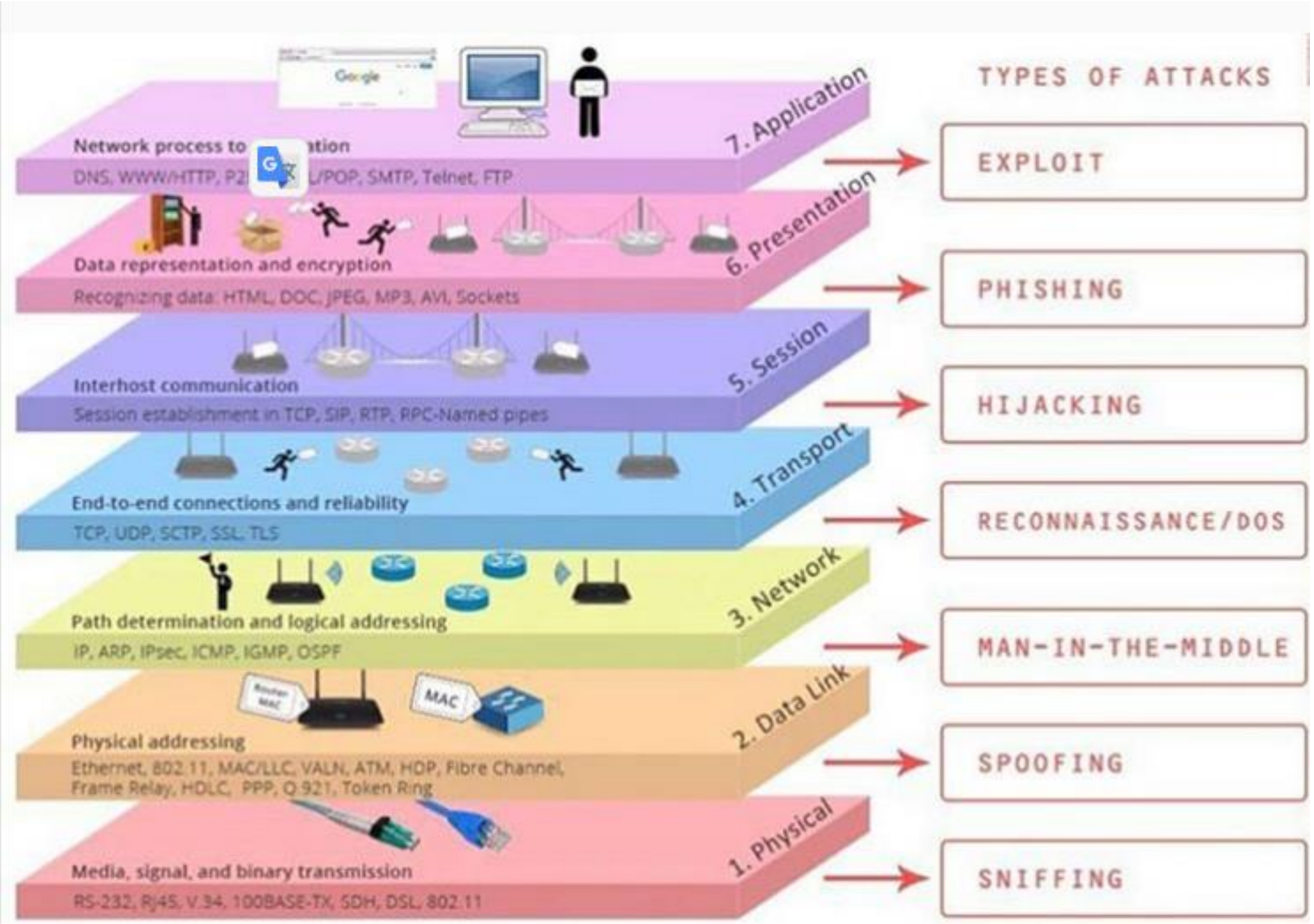
OSI, TCP/IP, security attacks, and forensic evidence


OSI Basic Reference Model
Protocols in Each Layer
TCP/IP Model
Data
APPLICATION
PRESENTATION
SESSION
Modbus, SEP2, DNP3, HTTP, IEC 61850, CIM, ICCP, BACnet, OpenADR, GOOSE
Compression an encryption protocols
NFS, SQL, SMB, RPC, P2P tunneling, SCP, SDP, SIP, H.323
APPLICATION
Data
Segments
TRANSPORT
TCP, UDP
TRANSPORT
Segments
Packets
NETWORK
IPv4/IPv6, ARP, IGMP, ICMP
INTERNET
Packets
Frames
DATA LINK
Ethernet
Bits
PHYSICAL
RS 232, UTP cables (CAT 5, 6), DSL, Optic fiber
NETWORK INTERFACE
Bits and Frames

Evidence you should collect

File, images, Software installed, etc.

	Layer	
OS	Application Layer	N/w access to Application e.g. Web Browser (IE, Mozilla Firefox, Google Chrome)
	Presentation Layer	Type of Data; HTTPS – Encryption Sevices
	Session Layer	Starts and Ends session and also keeps them isolated.
Network	Transport Layer	Defines Ports and Reliability
	Network Layer	Logical or IP addressing; Determines Best path for the destination.
	Data Link Layer	Switches MAC Addressing
	Physical Layer	Cable Network Interface Cards – Electric Signals


TYPES OF ATTACKS
7. Application
Network process to ation
DNS, WWW/HTTP, P2 L/POP, SMTP, Telnet, FTP
EXPLOIT
6. Presentation
Data representation and encryption
Recognizing data: HTML, DOC, JPEG, MP3, AVI, Sockets
PHISHING
5. Session
Interhost communication
Session establishment in TCP, SIP, RTP, RPC-Named pipes
HIJACKING
4. Transport
End-to-end connections and reliability
TCP, UDP, SCTP, SSL, TLS
RECONNAISSANCE/DOS
3. Network
Path determination and logical addressing
IP, ARP, IPsec, ICMP, IGMP, OSPF
MAN-IN-THE-MIDDLE
2. Data Link
MAC
Physical addressing
Ethernet, 802.11, MAC/LLC, VALN, ATM, HDP, Fibre Channel, Frame Relay, HDLC, PPP, Q.921, Token Ring
SPOOFING
1. Physical
Media, signal, and binary transmission
RS-232, RJ45, V.34, 100BASE-TX, SDH, DSL, 802.11
SNIFFING

Assignment

Requirements

- Using curl to sent request to the local web server
  - Capture the http requests and responses in image.log
  - Why can't you see the TCP segments?
- Using Chrome to sent requests from Win 10 and save the log file in image3.log.
  - insert any one photo to .html
  - Extract the photo

Make sure:

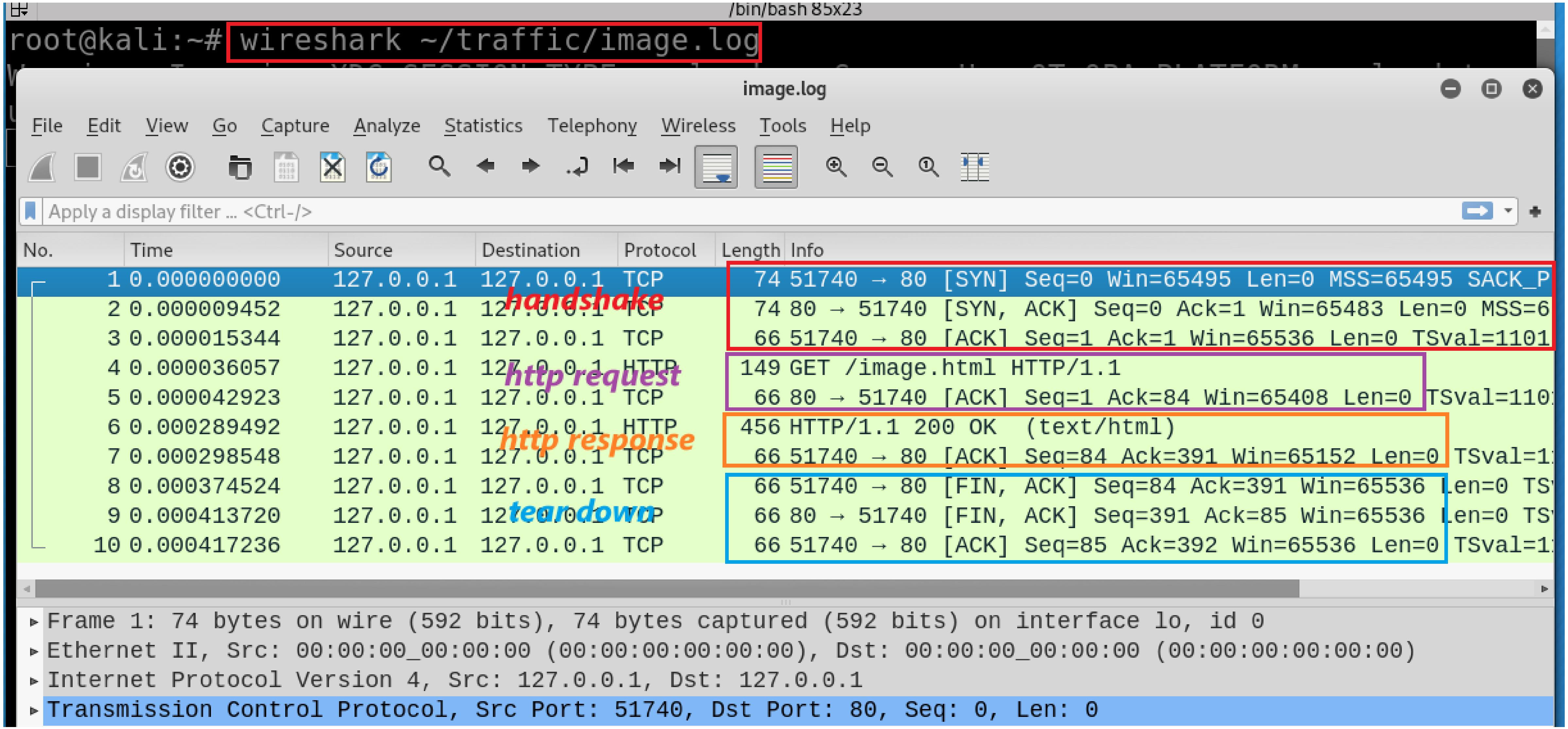
- service apache2 start
- has traffic folder

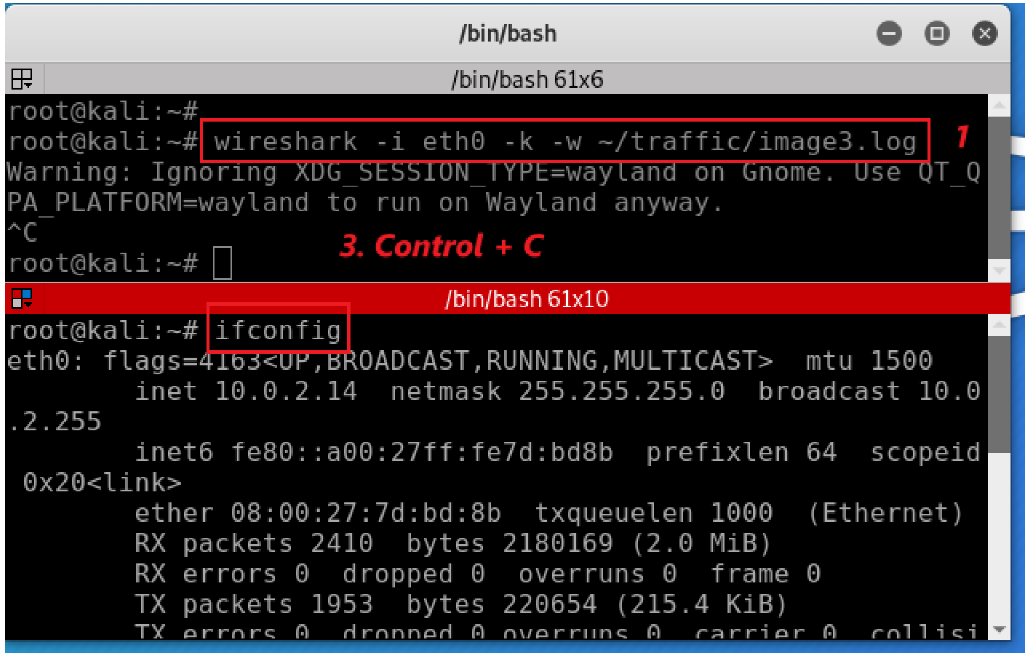
/bin/bash 85x8

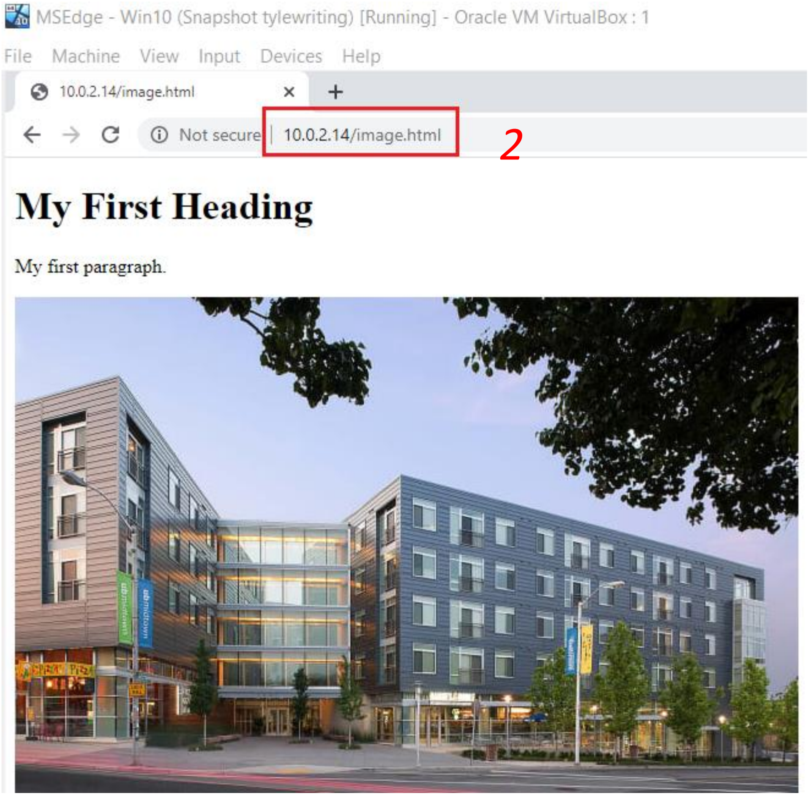
```
root@kali:~# rm -r  ~/.cache/mozilla/firefox/*default*     1
rm: cannot remove '/root/.cache/mozilla/firefox/*default*': No s
root@kali:~# wireshark -i lo -k -w ~/traffic/image.log     2
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_
un on Wayland anyway.
^C     4
root@kali:~#
```

/bin/bash 85x14

```
root@kali:~# curl 127.0.0.1/image.html     3
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

<img src="building.jpg" alt="UB"/>

</body>
</html>
root@kali:~#
```


/bin/bash 85x23
root@kali:~# wireshark ~/traffic/image.log
image.log
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 127.0.0.1 127.0.0.1 TCP 74 51740 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_P
2 0.000009452 127.0.0.1 127.0.0.1 TCP 74 80 → 51740 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=6
3 0.000015344 127.0.0.1 127.0.0.1 TCP 66 51740 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1101
4 0.000036057 127.0.0.1 127.0.0.1 HTTP 149 GET /image.html HTTP/1.1
5 0.000042923 127.0.0.1 127.0.0.1 TCP 66 80 → 51740 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TSval=110
6 0.000289492 127.0.0.1 127.0.0.1 HTTP 456 HTTP/1.1 200 OK (text/html)
7 0.000298548 127.0.0.1 127.0.0.1 TCP 66 51740 → 80 [ACK] Seq=84 Ack=391 Win=65152 Len=0 TSval=1
8 0.000374524 127.0.0.1 127.0.0.1 TCP 66 51740 → 80 [FIN, ACK] Seq=84 Ack=391 Win=65536 Len=0 TS
9 0.000413720 127.0.0.1 127.0.0.1 TCP 66 80 → 51740 [FIN, ACK] Seq=391 Ack=85 Win=65536 Len=0 TS
10 0.000417236 127.0.0.1 127.0.0.1 TCP 66 51740 → 80 [ACK] Seq=85 Ack=392 Win=65536 Len=0 TSval=1
handshake
http request
http response
tear down
Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 51740, Dst Port: 80, Seq: 0, Len: 0


/bin/bash
/bin/bash 61x6
root@kali:~#
root@kali:~# wireshark -i eth0 -k -w ~/traffic/image3.log
1
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_Q
PA_PLATFORM=wayland to run on Wayland anyway.
^C
root@kali:~#
3. Control + C
/bin/bash 61x10
root@kali:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.0.2.14 netmask 255.255.255.0 broadcast 10.0
.2.255
inet6 fe80::a00:27ff:fe7d:bd8b prefixlen 64 scopeid
0x20<link>
ether 08:00:27:7d:bd:8b txqueuelen 1000 (Ethernet)
RX packets 2410 bytes 2180169 (2.0 MiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1953 bytes 220654 (215.4 KiB)


MSEdge - Win10 (Snapshot tylewriting) [Running] - Oracle VM VirtualBox : 1
File Machine View Input Devices Help
10.0.2.14/image.html
Not secure
10.0.2.14/image.html
2
My First Heading
My first paragraph.